

Edge cases (context limit) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (catching it before it happens, instead of just after). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and forcing it on purpose](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and forcing it on purpose

Basic Version

Every model has a maximum amount of text it can look at in one call — the "context window." If you send more than that, the API won't quietly trim your message for you — it will refuse the whole call and give you back an error.

Your job here: make a message so long it goes past that limit on purpose, so you see the real error with your own eyes instead of just reading about it. The easiest way is repetition: a single word repeated tens of thousands of times is cheap to generate in Python and produces a huge token count fast.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Each model has a fixed token budget for the whole request (system + user + history + the reply it's about to generate). Send a request whose token count exceeds that budget, and the API raises an error rather than silently truncating anything on your behalf — truncating your input for you would be a dangerous, silent behavior for an API to have, since it could quietly drop something important without telling you.

`huge_input = "word " * 200_000` builds a string far larger than any current model's context window in a single line. Wrap the API call in `try: ... except openai.BadRequestError as e:` — a context-length problem surfaces as a `BadRequestError` (HTTP 400), not a special "context" exception of its own, so catching that specific type (not a bare `except:`) is what actually shows you which real exception class this is. Print `type(e)` and `str(e)` — the message text usually names the token counts involved directly, which is worth reading once for real.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Catching `BadRequestError` proves the limit exists, but it's a bad way to actually run a real app — you've already paid for the network round-trip to OpenAI's servers by the time you find out the request was always going to fail. A real chat feature wants to know *before* sending, so it can react intelligently (trim the oldest messages, summarize, or just tell the user) instead of always paying that cost to find out the hard way.

The real design question: **how do you count tokens locally, without an API call, using the exact same rules the model uses?** OpenAI publishes the tokenizer their models use as a separate library, `tiktoken` — the same text always produces the same token count, whether you count it locally or let the API count it for you and reject the request.

```
# context_limit_practice.py
import tiktoken

def count_tokens(text: str, model: str = "gpt-4o-mini") -> int:
    encoding = tiktoken.encoding_for_model(model)
    return len(encoding.encode(text))
```

Given a token count and a budget, sketch the decision yourself: if the count is over budget, do you refuse outright, or try to fix it (trim, summarize) and continue? There's no single right answer — write down which you'd pick for a chat app, and why, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both react to the limit *after* hitting it — building a huge input, sending it, and reading the resulting error. Advanced moves the check to *before* sending anything at all, using `tiktoken` to count tokens locally with the same rules the API uses, which turns "wait for the API to reject it" into "know instantly, for free, without a network call" — the difference between catching a mistake after it costs you a round-trip and never making the mistake in the first place.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
build a huge piece of text (a word repeated many, many times)

try:
    send it to the model
except (something went wrong):
    print what kind of error it was
    print the error's message
```

Here's almost the whole thing:

```
# context_limit_practice.py
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except Exception as e:
    print(type(e).__name__, str(e)[:200])
```

Expected output: something naming `BadRequestError` and a message mentioning the model's maximum context length and roughly how many tokens your message actually was.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
import openai
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    print(type(e).__name__)
    print(str(e))
```

Run it — then compare catching the *specific* error type against the bare `Exception` from `Basic`. Notice `openai.BadRequestError` is what you'd actually write in real code, since a bare `except` would also silently swallow a genuine bug in your own request-building code, not just this one expected failure.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
MAX_TOKENS = a safe number, below the model's real limit

function count_tokens(text, model) -> int:
    get the right tokenizer for this model
    return how many tokens the text encodes to

huge_input = "word " * 200_000
token_count = count_tokens(huge_input)

if token_count > MAX_TOKENS:
    print a message saying it's too long, and don't send it
else:
    send it normally
```

Turning that into real code — fill in the missing piece yourself:

```
# context_limit_practice.py
import tiktoken

MAX_TOKENS = 120_000 # leave headroom below the model's real limit

def count_tokens(text: str, model: str = "gpt-4o-mini") -> int:
    encoding = tiktoken.encoding_for_model(model)
    return len(encoding.encode(text))

huge_input = "word " * 200_000
token_count = count_tokens(huge_input)

# your turn: if token_count > MAX_TOKENS, print a clear refusal
# message and don't call the API at all; otherwise send it as normal
...
```

Expected output: Input is 200000 tokens – too long, refusing to send. (the exact count will vary slightly by tokenizer version), with no API call made at all — confirm this yourself by noticing the script finishes instantly instead of waiting on a network response.

Fill in the local check yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both discover the limit reactively — send first, catch the failure after. Advanced counts tokens locally with `tiktoken` *before* sending anything, so a request that was always going to fail never leaves your machine — cheaper, faster, and it gives you the chance to react (trim, summarize, refuse) instead of just reporting a failure that already happened.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)